Practice 8 — Permissions. Part A: Theory Answers

Student: Avazjon Islomov

Group: SE2-24

Question 1 — The permission workflow

In short: Before writing any code, first ask: "Do I even need a permission?"

The four decisions, in order:

1. Do I need a permission at all? Maybe another app or a system tool can do the job for me.
2. Which category is it? Install-time, runtime or special.
3. Declare it in the manifest with <uses-permission>.
4. Request it at the right moment (when the user uses the feature) and handle the answer (granted or denied).

Example without any permission: taking a photo with the system camera app (TakePicturePreview). The system camera app takes the photo, not my app, so my app does not need CAMERA. Another example is the Photo Picker for choosing a photo from the gallery.

Why this is the best option: there is no dialog, the user cannot deny it, so the feature always works. There is also less code and better privacy.

Question 2 — Three permission categories

In short: The category decides how a permission is granted, so it also decides how much code you write.

- Install-time (normal and signature): the system grants it automatically when the app is installed. You write only one line in the manifest.
- Runtime (camera, location, contacts…): the user taps "Allow" in a dialog while the app is running. You write the manifest line, a check, a request, and code for denial and rationale.
- Special (overlay, all files access…): the user turns on a switch in the Settings screen. You write the manifest line, open Settings with an Intent, and check again when the user returns.

So "one line in the manifest" is true only for install-time permissions.

Question 3 — Normal permissions

In short: Normal permissions are low-risk, so the system gives them automatically at install time.

- A normal permission does not touch private user data.
- The system grants it at install time, with no dialog, and the user cannot turn it off.
- So checkSelfPermission before every network request is useless: it is always granted.
- If you forget the manifest line, the app throws a SecurityException.

Three examples: INTERNET, ACCESS_NETWORK_STATE, VIBRATE.

Question 4 — Signature permissions

In short: A signature permission is given only to apps signed with the same key as the app that declared it.

The Bank app declares it like this:

```
<permission
    android:name="uz.ttpu.bank.permission.READ_ACCOUNTS"
    android:protectionLevel="signature" />
```

- Wallet is signed with the same company key, so it gets the permission automatically at install and can bind to AccountsService.
- The stranger app has a different key, so it does not get the permission. Binding fails with SecurityException.
- No dialog is shown in both cases. The system decides only by comparing the signing keys.

Question 5 — Install-time permissions

In short: Install-time permissions are not asked with a dialog — they are given when the app is installed.

- The user sees them on the app's page in the store (Google Play), before installing.
- The system grants them automatically at install time.
- There is no dialog while the app is running.

Two kinds belong here: normal permissions (for example INTERNET) and signature permissions (for example READ_ACCOUNTS from Question 4).

Question 6 — Runtime permissions and Android versions

In short: Runtime permissions appeared in Android 6.0 (API 23). Before that, everything was granted at install.

- On API 22 (Android 5.1) runtime permissions did not exist yet. The user accepted all permissions when installing the app. So CAMERA is granted at install, there is no dialog and the camera works.
- On API 23 and higher (for example Android 14) CAMERA is a runtime permission, so the user must allow it in a dialog.

The granted(kind, sdk) model shows this rule: the same runtime permission is granted "at install" if sdk < 23, and needs a "dialog at runtime" if sdk >= 23. Normal permissions are always "at install", special ones are always "in Settings".

Question 7 — Denial states

In short: After two denials Android stops showing the dialog — you must send the user to Settings.

1. Not asked yet — shouldShowRationale = false. Call launch(), the dialog appears.
2. Denied once — shouldShowRationale = true. First show an explanation ("We need the camera to take photos"), then call launch() again.
3. Denied twice — shouldShowRationale = false. The dialog will not appear any more, launch() returns "denied" immediately. Show an "Open Settings" button.

After two denials the user must be sent to the app's settings page (Settings.ACTION_APPLICATION_DETAILS_SETTINGS), where they can turn the permission on by hand.

Note: states 1 and 3 both have shouldShowRationale = false. The difference is that in state 3 we already asked before, and the app has to remember that.

Question 8 — CameraPermissionExample (Accompanist)

In short: One when with three branches covers all states of the permission.

- isGranted — runs when the user allowed the camera. The screen shows the camera content.
- shouldShowRationale — runs when the user denied once. The screen shows an explanation and a "Request again" button.
- The remaining state (else) — runs when we have not asked yet (or after two denials). The screen shows "Permission required" and a request button.

Why does the state survive recomposition? rememberPermissionState uses remember inside. The state object is created once and is not reset every time the UI redraws. It is also an observable state, so when the user answers, the UI redraws by itself.

What does ExperimentalPermissionsApi mean? This API is experimental and can change in future versions. @OptIn means "I know this and I accept it".

Question 9 — Without a library (Activity Result API)

In short: Android itself can request a permission with rememberLauncherForActivityResult and RequestPermission().

- rememberLauncherForActivityResult(ActivityResultContracts.RequestPermission()) creates a launcher. launcher.launch(CAMERA) shows the dialog, and the callback gets true or false.

- Read the current state: ContextCompat.checkSelfPermission(...)
- Should I show a rationale? ActivityCompat.shouldShowRequestPermissionRationale(...)

Comparison with Accompanist:

- The Activity Result API needs no extra library and is official and stable, but you write more code because you keep the state yourself.
- Accompanist needs an extra library and is experimental, but you write less code because isGranted and shouldShowRationale are ready for you.

Question 10 — Several permissions at once (location)

In short: RequestMultiplePermissions returns a Map<String, Boolean>: the permission name and whether it was granted.

- To read it, take the value by the permission name: result[ACCESS_FINE_LOCATION] == true.
- If fine is granted, the location is precise. If only coarse is granted, it is approximate. If none, there is no location.

Why only one dialog? Both permissions are in the same Location group. Since Android 12 the user sees one dialog and chooses "Precise" or "Approximate" there.

If the user denies both: the app must not crash. It should still work without location, for example let the user type a city by hand, and explain why location is useful.

Question 11 — Special permissions

In short: Special permissions have no dialog — the user turns them on in the Settings screen.

How they differ from runtime permissions:

- A runtime permission shows a system dialog, and the user taps "Allow".
- A special permission has no dialog. The app opens a Settings screen with an Intent, the user turns on a switch and comes back.
- Each special permission has its own check function.

The table, written as a list:

- SYSTEM_ALERT_WINDOW — allows drawing over other apps (floating windows). Check: Settings.canDrawOverlays(context)
- WRITE_SETTINGS — allows changing system settings, for example screen brightness. Check: Settings.System.canWrite(context)
- MANAGE_EXTERNAL_STORAGE — allows access to all files on the storage (file managers). Check: Environment.isExternalStorageManager()

- REQUEST_INSTALL_PACKAGES — allows installing other apps from APK files. Check: context.packageManager.canRequestPackageInstalls()
- ACCESS_BACKGROUND_LOCATION — allows getting location while the app is in the background. Check: ContextCompat.checkSelfPermission(context, ACCESS_BACKGROUND_LOCATION)

Question 12 — Check → open Settings → check again

In short: Settings sends nothing back, so we check the permission again every time the screen comes back (ON_RESUME).

The pattern:

1. Check — Settings.canDrawOverlays(context).
2. Open Settings — if it is not allowed, send the user to the Settings screen.
3. Check again on return — when the user comes back, read the value again.

Why no callback? We open Settings with a simple startActivity(intent). Settings does not return any result to our app, so our code does not know the user came back. In the old code remember read the value only once, so the old false stayed on the screen.

The fix: add LifecycleEventEffect(Lifecycle.Event.ON_RESUME) and read Settings.canDrawOverlays(context) again inside it. Now the Switch always shows the real state.

```
@Composable
fun OverlayScreen() {
    val context = LocalContext.current
    var canDraw by remember {
        mutableStateOf(Settings.canDrawOverlays(context))
    }

    // runs every time the user comes back to the screen
    LifecycleEventEffect(Lifecycle.Event.ON_RESUME) {
        canDraw = Settings.canDrawOverlays(context)
    }

    Switch(
        checked = canDraw,
        onCheckedChange = { openOverlaySettings(context) }
    )
}
```

Question 13 — Background location in two steps

In short: First ask for normal (foreground) location, and only after it is granted ask for background location.

- Step 1: request ACCESS_FINE_LOCATION / ACCESS_COARSE_LOCATION — location while the app is open.

- Step 2: only after step 1 is granted, request ACCESS_BACKGROUND_LOCATION — location while the app is closed. On Android 11+ this takes the user to Settings, where they choose "Allow all the time".

Why not together? If you request both in one launch, the system ignores the background request. Background access only makes sense when foreground access already exists, and step by step is also easier for the user to understand.

Why the SDK_INT >= Build.VERSION_CODES.Q check?
ACCESS_BACKGROUND_LOCATION was added in Android 10 (API 29, "Q"). On older versions it does not exist, and foreground location already includes background access, so there is nothing to request.

Question 14 — Permission groups

In short: Related permissions are joined in a group, and the system shows one dialog per group.

A permission group is a set of permissions about the same thing. The dialog is shown for the group, not for each permission. That's why 5 permissions gave only 3 dialogs:

- Location group: ACCESS_FINE_LOCATION and ACCESS_COARSE_LOCATION — 1 dialog
- Contacts group: READ_CONTACTS and WRITE_CONTACTS — 1 dialog
- Camera group: CAMERA — 1 dialog

Why not rely on groups? Google says groups can change in new Android versions. So the code must always request every permission it needs and check each one separately. Never think "I have one permission from the group, so I have the other one too".

Question 15 — Forgetting <uses-permission>

In short: If a permission is not in the manifest, the system will not even show the dialog.

What happens: launch(CAMERA) returns "denied" immediately, no dialog appears. If the code uses the camera anyway, the app throws SecurityException.

How declaration differs:

- Install-time: only <uses-permission> in the manifest, nothing else.
- Runtime: <uses-permission> in the manifest and a request in code.
- Special: <uses-permission> in the manifest and sending the user to Settings.

android:maxSdkVersion is used when a permission is needed only on old Android versions:

```
<uses-permission
    android:name="android.permission.WRITE_EXTERNAL_STORAGE"
    android:maxSdkVersion="28" />
```

From Android 10 (API 29) apps can save their own files without this permission (scoped storage), so we ask for it only up to API 28.

Question 16 — <uses-feature> and store filtering

In short: required="true" hides the app on devices without a camera; required="false" shows it everywhere, but then the code must check for a camera.

```
<uses-feature
    android:name="android.hardware.camera"
    android:required="false" />
```

- required="true" (the default) — the store hides the app on devices without a camera, for example Android TV.
- required="false" — the app is visible everywhere. Then the code must check packageManager.hasSystemFeature(PackageManager.FEATURE_CAMERA_ANY) and hide the camera button if there is no camera.

Note: only adding the CAMERA permission makes the store think the camera is required. That's why required="false" must be written explicitly.

The four results of the visibleOn model:

1. Required = true, device has a camera — the app is visible. Normal case.
2. Required = true, no camera — the app is hidden. This is the team's problem.
3. Required = false, device has a camera — the app is visible, the camera works.
4. Required = false, no camera — the app is visible, and the code hides the camera button.

Question 17 — One-time permission ("Only this time")

In short: "Only this time" works only while the app is in use — so never save "granted" forever, check again on ON_RESUME.

How it works (Android 11, API 30):

- For camera, microphone and location the user can choose "Only this time".
- The permission works while the app is open.
- It is revoked when the user leaves the app and it stays in the background for a short time (about a minute), or when the app is closed.

Why not cache it? A permission can be removed at any moment: one-time permission ends, the user turns it off in Settings, or auto-reset removes it. If we saved true once, the button looks enabled but the camera does not work.

The fix: use var with mutableStateOf and read the permission again inside LifecycleEventEffect(Lifecycle.Event.ON_RESUME).

```
@Composable
fun CameraScreen() {
    val context = LocalContext.current
    var hasCamera by remember { mutableStateOf(context.hasCamera()) }

    // read again every time the user comes back
    LifecycleEventEffect(Lifecycle.Event.ON_RESUME) {
        hasCamera = context.hasCamera()
    }

    // ...
}
```

Question 18 — Auto-reset and removing unneeded permissions

In short: Android removes permissions from apps you don't use, and an app can give back permissions it no longer needs.

(1) Auto-reset (Android 11+)

- If the user has not opened the app for a few months, the system removes its runtime permissions automatically.
- What the app should do: always check the permission before using it, and if it is gone, request it again in the normal way.
- How the user can turn it off: the app opens Intent.ACTION_AUTO_REVOKE_PERMISSIONS, and the user switches off "Remove permissions if app isn't used".

(2) revokeSelfPermissionOnKill (Android 13, API 33)

- The app gives back a runtime permission it does not need any more, for example READ_CONTACTS. It is removed when the app process is closed.
- Why it is good for the user: better privacy. The app keeps only what it really uses, and the user trusts it more.

Question 19 — Changes across Android versions

In short: Every new Android version gives the user more control.

- API 23 (Android 6.0): runtime permissions appeared — dangerous permissions are asked with a dialog while the app runs, not at install.
- API 30 (Android 11): one-time permissions ("Only this time") and auto-reset for unused apps appeared, and background location is given only through Settings.
- API 33 (Android 13): POST_NOTIFICATIONS became a runtime permission, and revokeSelfPermissionOnKill was added.

POST_NOTIFICATIONS:

- Before Android 13 notifications were allowed by default, no permission was needed.

- From Android 13 the app must ask the user with a dialog, like for the camera.
- Why not request it below API 33? On older versions this permission does not exist. The request does nothing and returns "denied", even though notifications actually work. So we wrap it in if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU).